

Xai — Intelligence Workspace

Product & Design Document

Live: <https://reco-ai-intelligence-workspace.vercel.app> **Figma:**

<https://www.figma.com/make/3U24rtY2ZBF17HZqTdjG1n/App-Builder?fullscreen=1> **Walkthrough**

video: <https://youtu.be/pTYRjmJRxA> **Repository:**

github.com/leotamim10/Rico_AI_Intelligence_Workspace

1. Product idea

Xai is an intelligence workspace. It ingests raw operational signal from across a company's stack — payments, product analytics, data warehouse, observability, CRM — structures the noise, surfaces what's anomalous, and routes the resolved insight into the automations that act on it.

Who it's for: the decision-maker, not the analyst. A head of operations, a VP of growth, a founder — someone who owns an outcome and needs to know what changed and what's already being done about it, without assembling five dashboards by hand.

What they get: one surface where 48,000+ daily signals collapse into a handful of decisions. Anomalies are surfaced in ~1.2s; automations fire before the person opens the tab. The product's promise is compression — from raw event stream to *decided action* — and the page is built to make that compression legible in a single scroll.

The voice throughout is product voice, not marketing voice: specific, short, declarative. "Ingest from 40+ sources." "Anomalies surfaced in 1.2s." Never "unlock the power of your data."

2. The narrative, decomposed into four sections

The brief's spine is a four-beat narrative:

raw data → structured intelligence → actionable insight → AI automations

Each beat becomes one page section, and the transition *between* beats is carried by motion rather than stated in copy:

Beat	Section	What the motion does
Raw data	Hero	A chaotic particle cloud resolves into a structured lattice as you scroll (<code>uProgress</code> 0 → 1).
Structured intelligence	Insight Flow	A pinned, scrubbed GSAP timeline draws a pipeline — sources feeding an analysis network — stage by stage: Ingest / Analyze / Generate.
Actionable insight	Dashboard	The abstract becomes concrete: a real product surface — KPIs, a hand-rolled chart, a signals table.
AI automations	Signature Moment	The same particle field clusters into grouped insight (<code>uProgress</code> 1 → 2) — the noise has become a small number of things worth acting on.

The particle field is the connective tissue. It is a *single system* used at both ends of the page — the hero drives the first half of its morph, the signature moment drives the second — so the visual argument ("chaos becomes structure becomes clustered insight") is literally one continuous transformation, not three unrelated animations.

3. Page structure & the scroll as a user flow

The page is a single continuous scroll. There is no nav, no CTA button, no marketing chrome — a headline-and-CTA layout would read as a landing page, which the brief explicitly penalizes. The scroll *is* the flow:

- **0%** — Hero. Chaos. The claim: "From raw signal to decided action." Two stat chips ground it in numbers. A scroll cue (geometry, not a fade) invites the first move.
- **~25%** — the hero's particle cloud finishes resolving into a lattice. The first beat has completed purely through scroll.
- **~40%** — Insight Flow pins. Scrolling now scrubs a timeline instead of moving the page; the pipeline draws itself, stage by stage, with an active indicator tracking progress.
- **~65%** — Dashboard. The product itself: sidebar, top bar, KPI row, chart, tabbed table. This is the "would it pass as a real SaaS screenshot" test.
- **~90–100%** — Signature Moment pins; the particle field clusters. The page closes on the payoff: insight, already in motion.

On touch / small screens the pinned sections unpin — each collapses to a single viewport and the particle field renders its resolved end-state — so the flow degrades to an honest vertical read instead of broken scroll-jacking.

4. Design system

Dark-first, one accent, restraint graded. Every value is a token defined once in `globals.css` and mirrored into the Tailwind theme — nothing is inlined.

Type. Three families, three weights (400/500/600 — never 700+). Inter for body, Geist for display headlines, Geist Mono for numerals. A six-step scale:

display	56 / 1.05 / 600 / -0.03em	body	15 / 1.55 / 400
h1	40 / 1.10 / 600 / -0.025em	small	13 / 1.50 / 400
h2	28 / 1.20 / 500 / -0.02em	mono	13 / 1.40 / 400 (tabular)

Why: a tight scale with heavy negative tracking on the large sizes reads as confident and technical; capping at weight 600 keeps it from feeling shouty. Numerals are tabular so metrics don't jitter.

Spacing. 8pt base, a fixed allowed set — 4, 8, 12, 16, 24, 32, 48, 64, 96, 128. Section padding 128px desktop / 64px mobile; max content width 1200px. *Why:* a closed scale is what makes spacing feel systematic rather than hand-tuned per element. If a value isn't on the scale, the scale is wrong — you change the scale, you don't inline the value.

Color. A near-black page (`#08090A`) with a four-step surface ramp (`surface-1..3`) instead of shadows for elevation, 1px low-contrast hairline borders, and exactly **one** accent (`#4F8FF7`) allowed on at most three elements per viewport. Positive/negative greens and reds are reserved for data. *Why:* restraint is the whole point — elevation by surface-step and hairline, not by drop-shadow, is what separates a product surface from a template. The single accent means the eye always knows where the important thing is.

Radius. `sm 6px` (buttons/pills), `md 10px` (cards), `lg 14px` (panels). No elevation shadows except a single soft one on floating overlays.

5. Motion rationale

All motion is centralized in `lib/motion/` — no easing or variant is defined inside a component. This is both a discipline (one place to tune the feel of the whole app) and a correctness guarantee (GSAP tweens and Framer variants read the *same* curves).

Four easings, each with a job:

Curve	Value	Communicates
out	[0.16, 1, 0.30, 1]	An element arriving — fast start, long settle. Entrances.
inOut	[0.65, 0, 0.35, 1]	A state changing — symmetric. Tab and content swaps.
micro	[0.4, 0, 0.2, 1]	A response to input — hover, focus, toggle.
spring	stiffness 260, damping 32	Physical motion — the <code>layoutId</code> indicators.

`linear` appears only on scroll-scrubbed timelines; the default `ease` is never used.

Duration bands map intent to time: micro 120–180ms (input feedback), element 300–450ms (a card or tab), section 600–900ms (a section arriving), and `scene` — scrubbed, where the "duration" is scroll distance.

The rule every animation is held to: *if you can't say what it means, delete it.* Concretely —

- **No fade-only transitions.** Every entrance moves geometry (a rise, a slide, a scale-settle), because opacity alone is a shallow animation.
- **The particle morph is the argument, not decoration.** Chaos → structure → cluster is the product thesis rendered literally. It runs on the GPU (a single `uProgress` uniform, three baked position sets, interpolation in the vertex shader) so 6000 points morph at 60fps.
- **Scroll values are damped, never assigned.** Scroll sets a target; a lerp (~0.09) eases toward it, so scrubbing feels weighted.
- **Entrances fire once;** the only looping motion is the particle field's idle breathing drift.

6. Trade-offs & what I'd do with more time

This was built to a two-day constraint, and the strategy was build-first, document-second — the Figma is reverse-engineered from working code rather than designed up front, which turns a design exercise into a mechanical one and removes a whole class of design-vs-implementation drift. Stated plainly, here is what that constraint cost and what I'd spend more time on:

- **Perf is measured; the fps profile is the gap.** A Lighthouse desktop pass on the production build scores Performance 99, Accessibility 100, Best Practices 100, with CLS 0 (LCP 0.9s, TBT 10ms) — and CLS and accessibility are hardware-independent, so those numbers hold anywhere. 60fps holds by construction (GPU morph, parked render loop, transform-only DOM animation), but a full Chrome DevTools frame-timeline profile on real hardware is still outstanding — this build environment's software GL renderer makes on-device frame timing unreliable. First follow-up: profile on a mid-tier laptop and a real phone.
- **Safari is audited and mitigated, not yet device-verified.** WebGL and `backdrop-filter` break in Safari first, so I audited for them: no `backdrop-filter` /glassmorphism, no version-sensitive CSS (`color-mix` , `oklch` , `:has`), Safari-safe WebGL context flags, and a correct sticky pattern. The one real finding — iOS Safari's `100vh` counting the collapsible toolbar — is fixed

by switching the hero and signature viewport containers to `100dvh` (`h-dvh`). A real-device Safari/iOS pass is the remaining step; the Linux build environment can't run WebKit.

- **Mobile is functional, not crafted.** Small screens unpin and reduce particle count so nothing is *broken* — but a phone deserves its own choreography, not just a de-risked desktop. With more time the signature moment would get a touch-native gesture, not a static end-state.
- **The dashboard is one workspace tab deep.** The nav and secondary tabs are plausible but inert; a fuller build would wire at least the tab content transitions to real mock datasets per tab.
- **No automated tests.** For a visual, motion-heavy single page inside two days, manual + browser verification was the right call — but I'd add visual regression snapshots and a reduced-motion assertion before iterating further.

None of these are surprises — they're the deliberate edges of a two-day scope, drawn on purpose so the four sections, the deployment, and the particle centerpiece could all land properly.

Figma: [App Builder workspace](#) — shared, view access for anyone with the link.